① CSci 2041
13 Feb 26

Announcement - videos

Higher-order functions -
- functions that take other functions as arguments.
- Classic examples
  - map
  - filter
  - reduce

Anonymous functions - functions with no names.

map f [e_0 ; e_1 ... ; e_{n-1}] ⇒ [f e_0 ; f e_1 ... ; f e_{n-1}]

↑ function 1 arg. ↑ list

map add1 [2 ; 3 ; 5 ; 1] ⇒ [3 ; 4 ; 6 ; 2]

```
let map func objects =
  let rec mapping objects =
    match objects
    with [] -> [] |
         head :: tail -> (func head) ::
                         (mapping tail)
in mapping objects ;;
```

tail recursive?
<u>Not</u> tail recursive.

↑ not a tail call

$\underbrace{('a \to 'b)}_{\text{func}} \to \underbrace{'a\ list}_{\text{objects}} \to \underbrace{'b\ list}_{\text{returned value}}$

parenthes are needed!

we can think of some functions as copying lists & making changes as we go.

```
let add1 number = number + 1 ;;
```

why do this? we often want to make copies of lists with changes of elements.

(2) CSCI 2041
13 Feb 26

<u>map</u>: like a function with parts missing, waiting to be filled in. Part to be filled in is <u>func</u>.

We used <u>add1</u> only once, why give it a name?

Anonymous function – function with no name.

$$(\text{fun } \underbrace{p_1\ p_2 \cdots p_n}_{\text{parameter names patterns}} \rightarrow e)$$

expression (as complicated as we want)

Returns a function that takes n arguments for $p_1, p_2 \cdots p_n$ & returns the value of e.

(fun number -> number + 1) Anonymous version of add1

map (fun number -> number + 1) [2; 3; 5; 1] ⇒ [3; 4; 6; 2]

we need not give this function a name.

Use this when we have a function passed to another function, used only once (a few times).

filter p [e_0; e_1 ⋯ ; e_{n-1}] ⇒ a new like the original except that the only e_i's that remain are those where p e_i is true.

predicate
one argument
returns bool

③ CSci 2041
13 Feb 26

```
let filter pred objects =
  let rec filtering objects =
    match objects
    with [] -> [] |
         head :: tail ->
           if pred head
           then head :: (filtering tail)
           else filtering tail
  in filtering objects ;;
```

ignorant people write pred head = true
Never do that.

not a tail call

tail call

(((b = true) = true) = true) = true ... (joke)

$$\underbrace{('a \to bool)}_{pred} \to \underbrace{'a\ list}_{objects} \to \underbrace{'a\ list}_{\text{returned value}}$$

Tail recursive? yes & no

```
filter (fun number -> number mod 2 = 0)
```

To

[1 ; 2 ; 3 ; 4 ; 5 ; 6] ⇒

[2 ; 4 ; 6]

tests if number is even.

keep only members of the list for which pred returns true.

(4) CSci 2041
13 Feb 26

what's this for?

reduce f [e_0; e_1 ...; e_{n-1}] e ⇒

2 arguments

f e_0 (f e_1 (f e_2 ... (f e_{n-1} e) ...))

uses f to reduce the e_i's to a single value.

(rightReduce) fold or rightfold ?

reduce f [] e ⇒ e

```
let reduce func objects defaultObject =
  let rec reducing objects =
    match objects
    with [] -> defaultObject |
         head :: tail -> func head (reducing tail)
  in reducing objects ;; ← tail call
```

NOT a tail call

$\underbrace{('a \to 'b \to 'b)}_{\text{func}} \to \underbrace{'a\ list}_{\text{objects}} \to \underbrace{'b}_{\text{default Object}} \to \underbrace{'b}_{\text{returned value}}$

Tail recursive? <u>No</u>.

reduce (+) [1; 2; 3] 0 ⇒ 6

this is the name of the int add function

Adds the elements in a list of ints.

int multiply function

reduce (␣*␣) [1; 2; 3] 1 ⇒ 6

use this to perform various operations on a list depending on <u>func</u>.